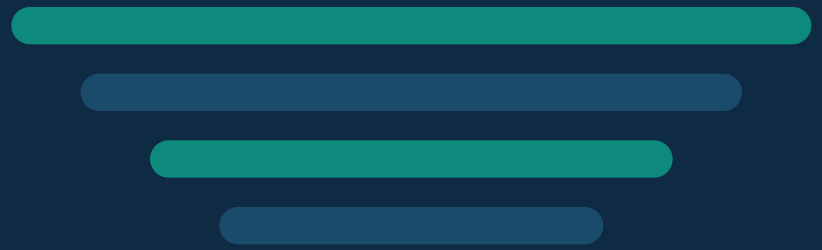
Parallelism and OpenMP

Slide summary: basic concepts, directives and performance evaluation



Organized from class notes

Outline

1 **Parallelism**
what it is, goal, motivation and challenge — slide 3

2 **Processes and threads**
basic concepts and multithreading — slides 4–6

3 **Shared × distributed memory**
two architecture models — slide 7

4 **OpenMP and fork-join**
API, components and execution model — slides 8–9

5 **Region and task directives**
parallel, parallel for, task, single, taskwait — slides 10–13

6 **Synchronization directives**
sections, barrier, master, critical — slides 14–15

7 **The reduction clause**
all-to-one operations without race conditions — slides 16–17

8 **Performance evaluation**
granularity, speedup and efficiency — slides 18–22

Quick review: slide 23 has the directive map and the formulas (G, S, E). Tip: click an outline item to jump straight to it.

What is parallelism?

- **Technique** that splits a computation into parts that can run **simultaneously** on multiple cores or processors
- **Goal:** reduce execution time or increase the amount of data processed in the same time interval
- **Advantageous** for problems with a large volume of computation and for problems that can be decomposed into independent tasks, or tasks with few dependencies

Motivation

Potential reduction in execution time through parallelism, combined with the wide availability of **multicore** processors in today's computing devices.

Challenge

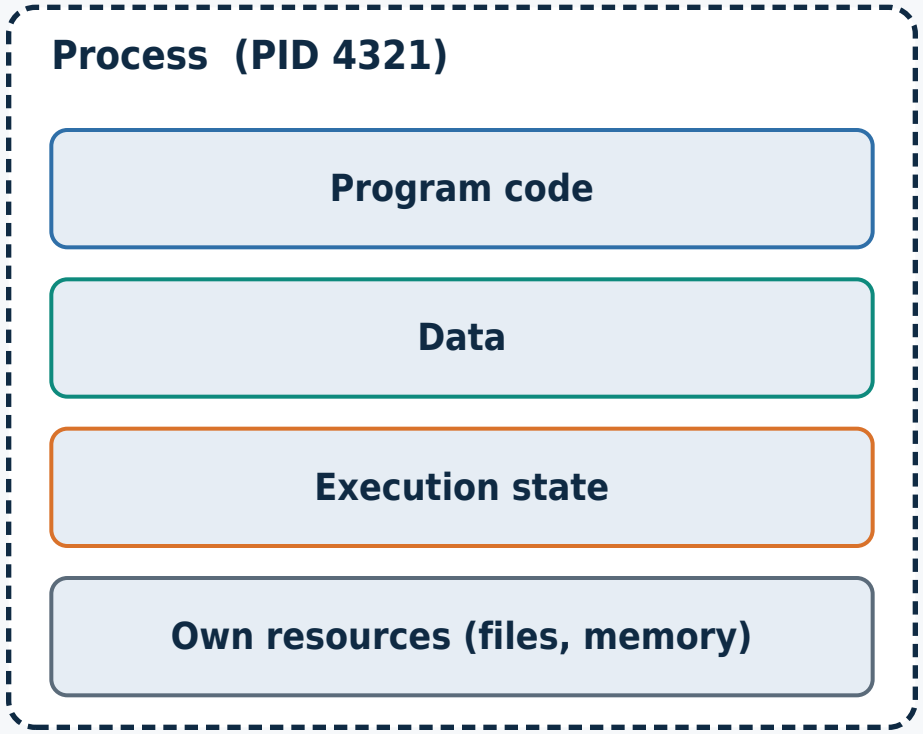
Not every developer knows the fundamentals of parallel computing. Even among those who do, applying parallelism **efficiently** is still a challenge.

Overhead = extra cost you pay to make something work (e.g., creating threads, synchronizing, communicating). When the overhead is larger than the gain, parallelism does not pay off.

Basic concepts: processes

- An instance of a **running program**
- Created and managed by the **operating system**
- Associated with an **owner user**
- Has its own code, data, execution state and **resources**
- **PID** = process number (identifier)
- The Linux command `top` shows all active processes

> **e.g.:** when I click to open Chrome, what happens is that I ask the OS to create a process that runs Chrome.

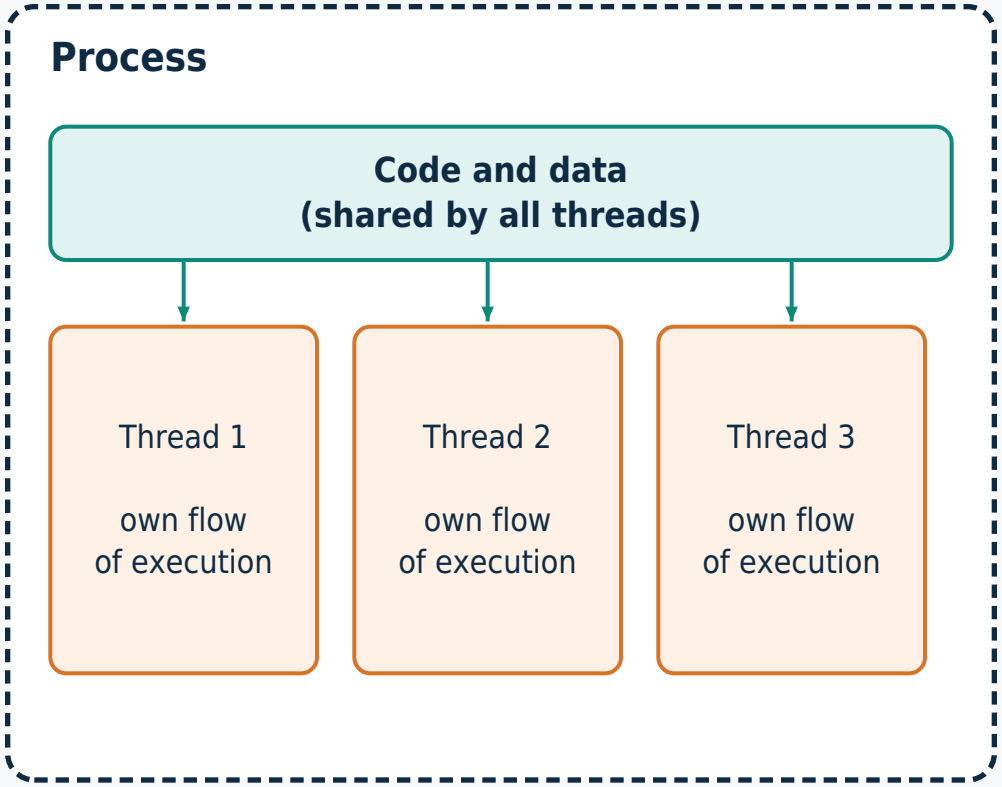


the OS creates and manages the process

Basic concepts: threads and multithreading

- **Thread:** a flow of execution that belongs to a process
- **Multithreading:** two or more threads running within the same process
- Threads **share** the process resources, such as code and data
- Each thread has its own **independent flow of execution**

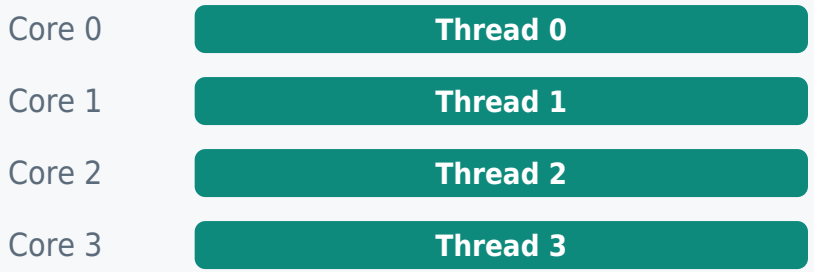
> **e.g.:** a multithreaded process, such as a multithreaded web server (one thread handling each request).



Advantages of threads

- They share the **address space** and the process resources, which makes communication between them easier. But care is needed so that one does not interfere with another, and for that there are **rules** (synchronization)
- **Lower creation and termination costs** than processes
- They make better use of the parallelism of **multicore** processors
- They let you parallelize parts of a sequential flow, **keeping the rest sequential**

> **BEST CASE:** put one thread on each core. It is much faster because there is dedicated processing. The programmer is expected to have split the work so that every thread gets the **same load** (load balancing).



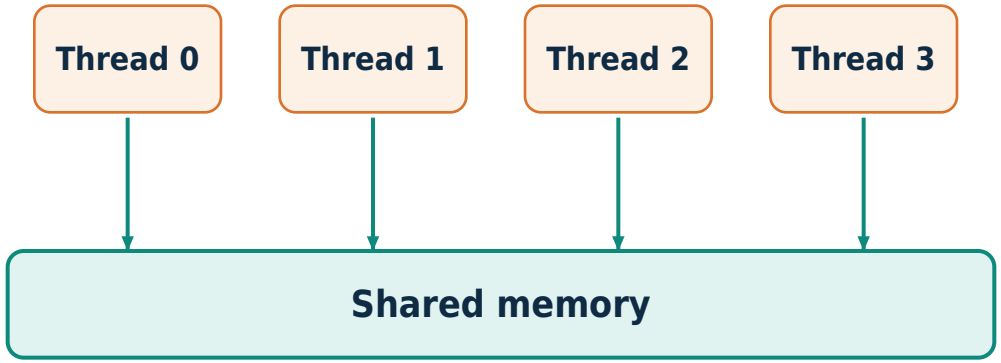
same load on every thread

Shared × distributed memory

Shared memory

Threads access the **same memory space**, communicating through **shared variables**.

OpenMP model (one multicore node).

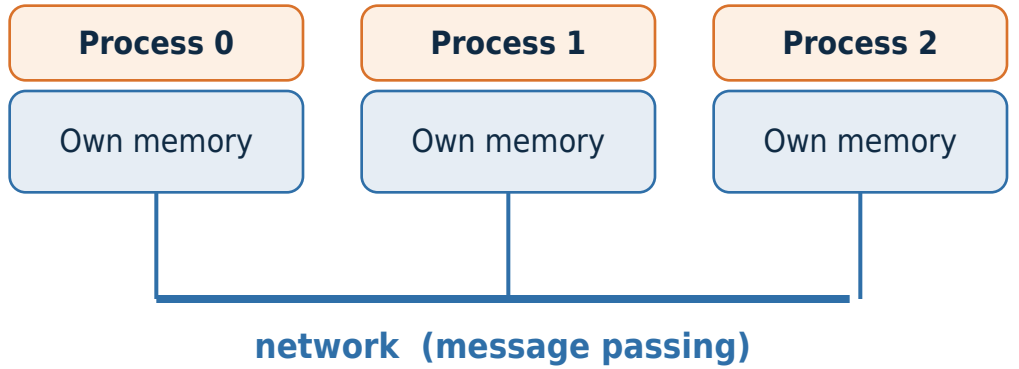


read and write the same variables

Distributed memory

Each machine has **its own memory**. The processes of the parallel application communicate by **message passing over the network**.

Typical cluster model (e.g., MPI).



each process only sees its own memory

OpenMP

- **API** for parallel programming on **shared-memory** architectures
- Supported by **C, C++ and Fortran** compilers
- Uses the **fork-join** model (task splitting) for parallel execution
- Made up of three parts:

Compiler directives

instructions that tell the compiler to parallelize a section

```
#pragma omp parallel
```

Library routines

functions called inside the program

```
omp_get_thread_num()
omp_get_num_threads()
```

Environment variables

configure execution outside the code

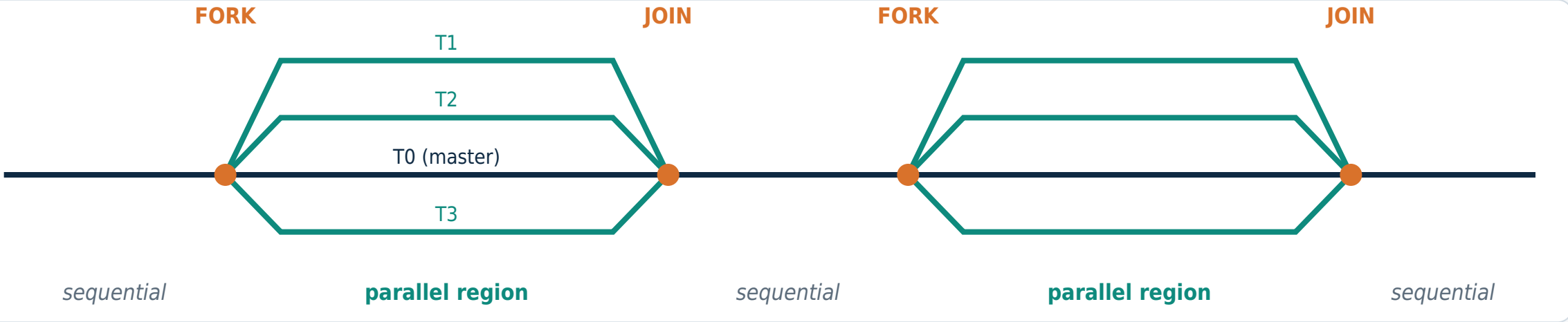
```
OMP_NUM_THREADS=4
```

How to compile and run (gcc):

```
gcc -fopenmp program.c -o program   // -fopenmp enables OpenMP
OMP_NUM_THREADS=4 ./program         // runs with 4 threads
```

Fork-join model

- Execution starts with **a single thread**: the master (thread 0)
- **Fork:** on entering a parallel region (`#pragma omp parallel`), the master creates a **team of threads**; all of them run the region in parallel
- **Join:** at the end of the region there is an **implicit barrier**. The threads synchronize and only the master continues
- After the join, the flow becomes **sequential** again until the next parallel region



The parallel and parallel for directives

omp parallel

Creates a **parallel region** in which **all** threads run the same block of code.

```
#pragma omp parallel
{
    printf("Hello from thread %d\n",
           omp_get_thread_num());
}
```

With 4 threads: **4 lines** printed (order is not guaranteed).

omp parallel for

Creates a parallel region and **distributes the iterations** of a loop among the threads.

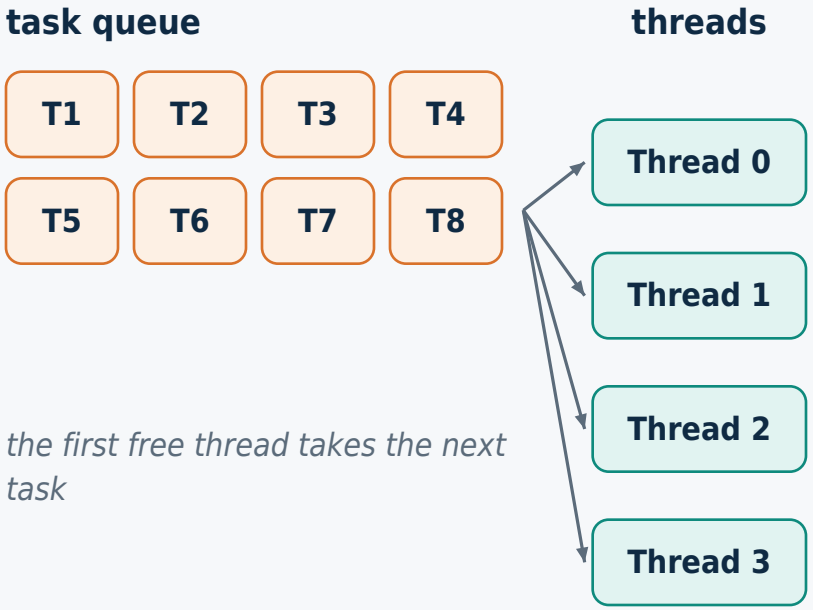
```
#pragma omp parallel for
for (int i = 0; i < 8; i++)
    c[i] = a[i] + b[i];
```

With 4 threads and static scheduling: each runs ≈ 2 iterations (T0: i=0,1 · T1: i=2,3 · …).

Note: in `parallel for`, the iterations must be **independent** of each other (one cannot depend on the result of another).

The omp task directive

- Creates a **task** that can be run by **any thread** in the team
- Lets you define units of work whose execution can happen **independently**
- Especially useful when tasks are **created dynamically** during execution
- How it works: I have a set of tasks created with `omp task`; the available thread picks one of them
- If there are **more tasks than threads**: each thread takes one task, and whichever finishes first takes the next one, and so on



IMPORTANT: creating a task does **not guarantee** that it will run immediately or in parallel. That is decided by the **OpenMP runtime**.

The omp single directive

- Specifies that a block of code is run by **a single thread**
- The thread that runs the block **does not have to be the master**
- Can be used so that **only one thread creates the tasks** of a parallel region
- Guarantees that two threads **do not run the same piece** at the same time
- There is an implicit barrier at the end of `single` (unless `nowait` is used); while waiting, the other threads can run the tasks that were created

Classic pattern: parallel + single + task

```
#pragma omp parallel
{
    #pragma omp single
    {
        for (int i = 0; i < N; i++) {
            #pragma omp task
            process(i);
        }
    }
}
```

> **Why single?** without it, **every thread** would create the N tasks, and the work would be repeated several times.

The omp taskwait directive

- Imposes a **synchronization point**: the task waits for its **child tasks** to finish before continuing
- It **slows** the program down, but in many cases it is necessary (e.g., when I need the children's results)
- Tasks are especially useful in **recursive algorithms**, where new units of work are created during execution

> **Link to granularity:** in very deep recursion (e.g., `fib(2)`), the task is so small that the overhead exceeds the work. A common fix is a cutoff: below it, continue sequentially.

e.g.: recursive Fibonacci with tasks

```
int fib(int n) {
    int x, y;
    if (n < 2) return n;
    #pragma omp task shared(x)
    x = fib(n - 1);
    #pragma omp task shared(y)
    y = fib(n - 2);
    #pragma omp taskwait  // waits for the 2 children
    return x + y;
}

int main() {
    int r;
    #pragma omp parallel
    #pragma omp single    // only 1 thread starts
    r = fib(30);
}
```

The sections and barrier directives

omp parallel sections

Splits the code into sections (`section`), each run by **a single thread**. Useful for **different, independent** jobs.

```
#pragma omp parallel sections
{
    #pragma omp section
    taskA();
    #pragma omp section
    taskB();
}
```

taskA and **taskB** run at the same time, on different threads.

omp barrier

Imposes a **synchronization point**: each thread **waits** until all the others reach it.

```
#pragma omp parallel
{
    phase1();
    #pragma omp barrier
    phase2();   // only starts when
                // all finish phase1
}
```

Used when phase 2 **depends** on the phase 1 results of all threads.

The master and critical directives

omp master

Specifies that a block of code is run **only by the master thread** (thread 0). The other threads **do not wait** (there is no implicit barrier).

```
#pragma omp parallel
{
    #pragma omp master
    printf("only thread 0 prints\n");
}
```

Difference from `single`: in `master` the thread is **always 0**.

omp critical

Creates a **critical region**: only **one thread at a time** runs it. If another tries to enter while it is in use, it **waits until it is released**.

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    #pragma omp critical
    total += v[i];  // 1 thread at a time
}
```

Prevents a **race condition**, but serializes the section. For sums, prefer **reduction** (next slide).

The reduction clause

- Applied to **all-to-one** operations: many values are combined into **a single result** (sum, product, maximum...)
- e.g.: a **shared sum**. With `reduction`, each thread works with a **local copy** of the sum variable; at the end of the parallel region, the local values are **combined** and the result is stored in the shared variable

> **What is the problem?** if all threads do `sum += v[i]` on the **same** variable, two threads may read the same old value and one update is lost (a **race condition**).

How reduction solves it (3 steps):

1 Fork

each thread gets a private copy, initialized with the operator's **identity element** (0 for +, 1 for ×)

2 Loop

each thread accumulates only into **its own copy**, with no communication or synchronization during the loop

3 Join

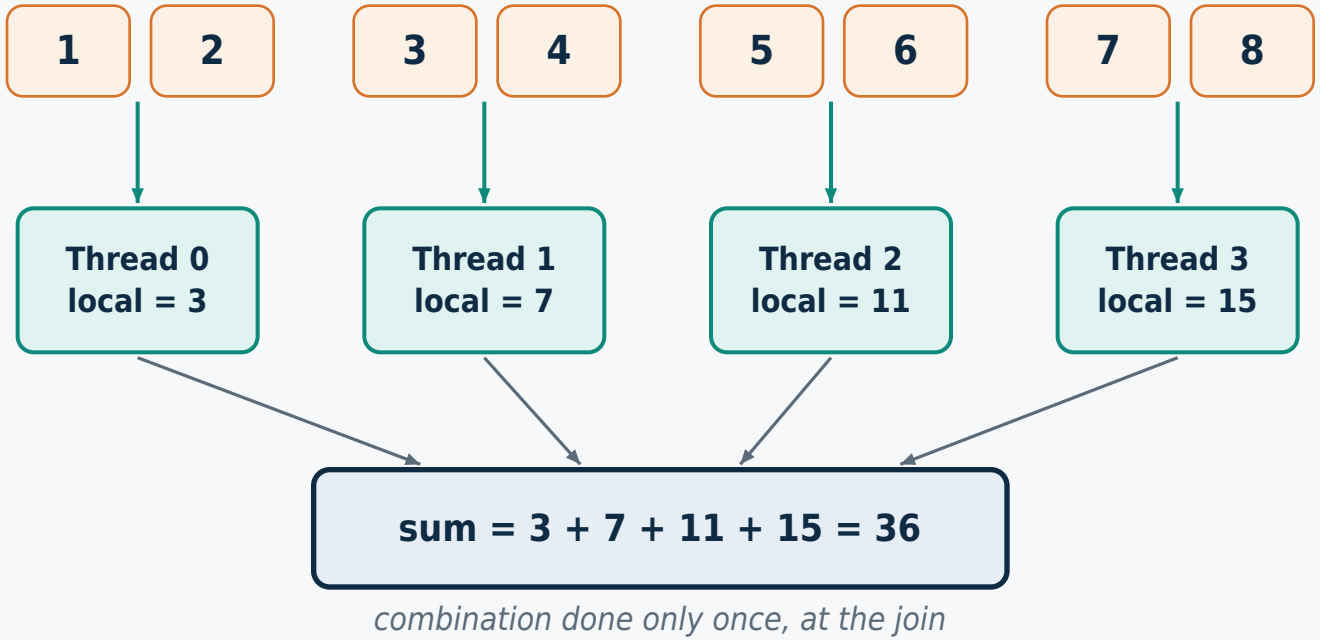
the local copies are **combined** with the operator and the result is stored in the original variable

```
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++)
    sum += v[i];
```

Operators: `+` `*` `-` `&` `|` `^` `&&` `||` `min` `max`

Reduction in practice

e.g.: sum of v = [1, 2, 3, 4, 5, 6, 7, 8] with 4 threads



combination done only once, at the join

Each thread only touches its own copy,
so there is no race condition during the loop.

Synchronization happens only once,
when the local copies are added up.

	critical	reduction
Synchronization	at every iteration	once, at the end
Result	correct	correct
Performance	serializes the section (slow)	scales well with more threads

Performance evaluation: granularity

- **Granularity:** the ratio between the amount of **computation** a task performs and the amount of **communication and synchronization** needed to integrate it with the execution of the other tasks
- The **higher** the ratio, the **higher** the granularity (coarse); the **lower** it is, the lower the granularity (fine)
- Appropriate granularity helps **reduce the overhead** of communication/synchronization and improves the **performance** and **scalability** of the parallel system

$$G = \frac{T_{comp}}{T_{comm + sync}}$$

T_{comp} **computation time (useful work of the task)**

$T_{comm + sync}$ **communication + synchronization time**

high G = coarse granularity · low G = fine granularity

Granularity: what does this ratio tell me?

- The ratio answers a simple question: **for every 1 unit of time spent on overhead, how many units do I spend doing real work?**
- G = 20: **20** of work for every **1** of overhead. Almost all the time is useful
- G = 0.4: more overhead than work (only 0.4 of work for every 1 of overhead)
- The ratio also gives the **fraction of time that is actually useful**: `G / (1 + G)`

> **e.g. (bricklayers):** four bricklayers building a wall. If after every brick they have to stop and tell the others (synchronize), they spend more time talking than building: **fine granularity**. If each gets a whole section of the wall and they only talk at the end: **coarse granularity**.

Fine granularity (low G)

Small tasks, lots of communication/synchronization. **Overhead dominates** and the gain disappears. On the plus side, the load is well distributed.

Coarse granularity (high G)

Large tasks, little overhead. Risk: **load imbalance** (one thread finishes early and sits idle) and fewer tasks to distribute.

Granularity: how to calculate it

- **1.** Estimate/measure `T_comp`: the useful computation time of a task
- **2.** Estimate/measure `T_ovh`: the communication + synchronization time tied to it (creation, waits, barriers, locks)
- **3.** Compute `G = T_comp / T_ovh` and the useful fraction `G / (1 + G)`

Task A
T_comp = 100 ms · T_ovh = 5 ms → G = 20



Task B
T_comp = 2 ms · T_ovh = 5 ms → G = 0.4



> **Reading:** in A (G = 20) the useful fraction is 20/21 ≈ **95%**: worth parallelizing. In B (G = 0.4) it is 0.4/1.4 ≈ **29%**: the overhead eats the gain, so it makes sense to **group tasks** to increase granularity.

How to increase granularity in OpenMP:

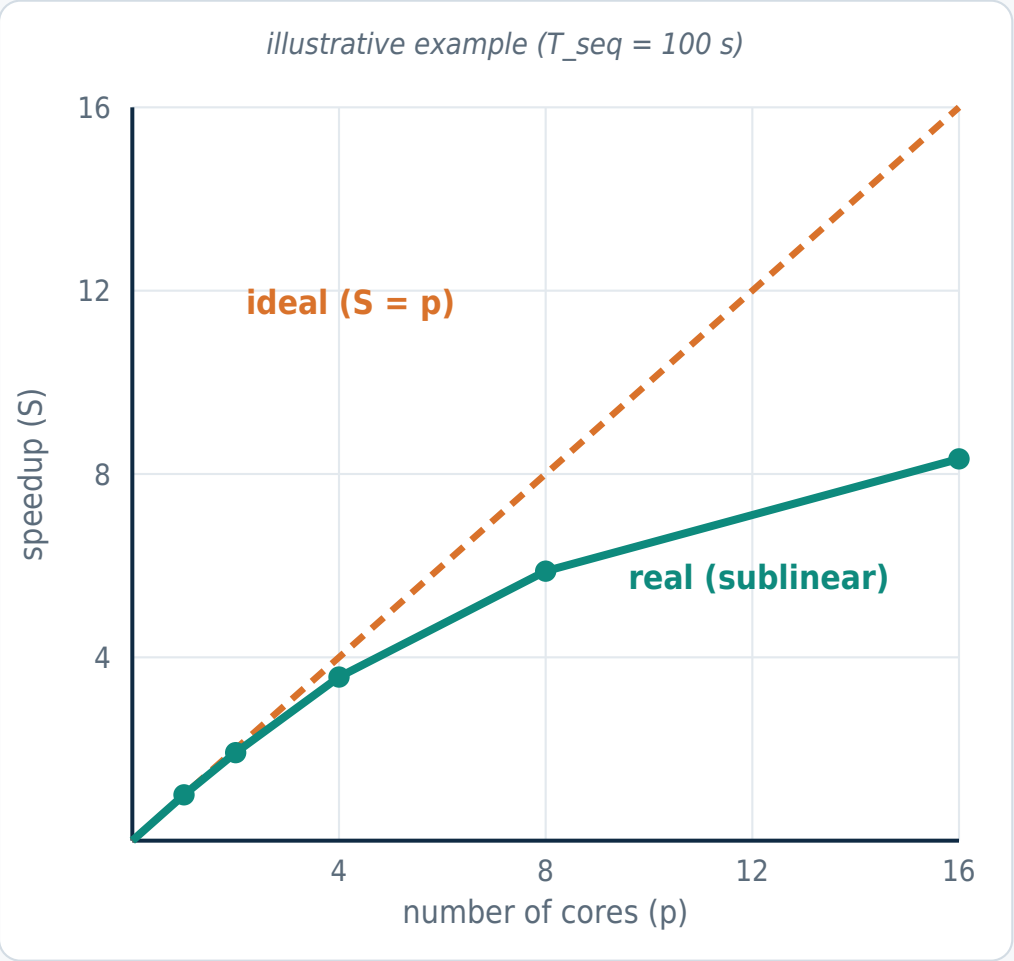
- `parallel for` with larger blocks of iterations (`schedule(static, chunk)`)
- Recursive tasks with a cutoff: below it, continue sequentially
- `reduction` instead of `critical` inside the loop

Speedup

- A metric that expresses the **performance gain** obtained by parallelizing an application
- `S = T_seq / T_par` (sequential time ÷ parallel time)
- **S > 1:** performance gain · **S < 1:** performance loss
- **Ideal** speedup grows linearly with the number of processors (S = p)
- In practice, **real** speedup grows **sublinearly**, because of communication and synchronization costs

e.g.: `T_seq` = 100 s; with 4 cores, `T_par` = 28 s → S = 100/28 ≈ **3.57**.

Amdahl's law: the sequential fraction f (the part that cannot be parallelized) limits the speedup:
`S ≤ 1 / (f + (1 − f) / p)`



Efficiency

- A metric that relates the **speedup** obtained to the **number of cores** used
- `E = S / p = T_seq / (p × T_par)`
- Efficiency **close to 1** (100%): **high utilization** of the cores
- Low efficiency: cores sitting idle or spending time on overhead

> **Trend:** as p grows, efficiency usually **drops**, because overhead grows and the sequential part weighs more.

p	T_par (s)	S	E	
1	100	1.00	1.00	
2	52	1.92	0.96	
4	28	3.57	0.89	
8	17	5.88	0.74	
16	12	8.33	0.52	

e.g.: T_seq = 100 s. Bar = efficiency (green ≥ 0.8)

Summary: directive map and metrics

Directive / clause	What it does
`parallel`	creates a parallel region; all threads run the same block
`parallel for`	parallel region + distributes loop iterations among the threads
`task`	creates a task that any thread in the team can run
`single`	block run by a single thread (any one)
`taskwait`	the task waits for its child tasks to finish
`parallel sections`	each code section is run by one thread
`barrier`	all threads wait for each other
`master`	block run only by the master thread (thread 0)
`critical`	critical region: one thread at a time
`reduction(op:var)`	local copy per thread + final combination (all-to-one)

Granularity
$G = T_{comp} / T_{ovh}$

Speedup
$S = T_{seq} / T_{par}$

Efficiency
$E = S / p$